

Diagnóstico VRTX Sentinel Audit

Versión: v1.0

Autor: Equipo Técnico VRTX Sentinel

Fecha: 2026-08-02

Resumen ejecutivo

VRTX Sentinel Audit es un SaaS orientado a seguridad operativa que combina telemetría, análisis automatizado y capacidades de IA para mejorar detección y respuesta. El estado actual del repositorio demuestra una base funcional para ejecución local, validación con stubs HTTP y pipeline mínimo de seguridad. También existe una ruta clara de hardening para evolucionar a staging y producción con controles de secretos y gobernanza. Conclusiones clave: - El host local arranca y expone endpoints stubbed para validación rápida. - Existe base CI con lint, test, SAST y control de CVEs críticos. - Hay documentación de seguridad (Key Vault + Managed Identity) y plan modular de 2 semanas. - Falta evidencia local consolidada de DAST/SAST exportada en release/security-reports. Recomendaciones prioritarias: 1. Activar protección de rama y checks requeridos en GitHub. 2. Completar reportes de seguridad en release/security-reports antes de producción. 3. Integrar secretos exclusivamente vía Key Vault + Managed Identity. 4. Mantener `ENABLE_LLM=false` y `ENABLE_DECEPTION=false` en etapas tempranas.

Índice automático

1. Objetivos y público	3
1.1 Objetivos del SaaS	3
1.2 Público objetivo	3
1.3 Casos de uso prioritarios	3
2. Funcionalidades y utilidades	3
2.1 Catálogo funcional	3
3. Arquitectura y tecnología	5
3.1 Diagrama textual	5
3.2 Stack tecnológico	5
3.3 Integraciones y adaptadores	5
4. Seguridad, pruebas y evidencia	5
4.1 Evidencia SAST/SCA/DAST disponible	5
4.2 Smoke tests y E2E (estado local)	6
4.3 Riesgos críticos y mitigación priorizada	6
4.4 Runbook de secretos y checklist hardening	6
5. Cómo aplicar en una empresa	6
5.1 Guía de integración por etapas	6
5.2 Requisitos operativos y roles	6
5.3 Políticas de deception	6
6. Diferenciadores e innovación	7
7. Plan de despliegue y roadmap corto plazo	7
7.1 Checklist staging/producción	7
7.2 Roadmap 2 semanas	7
7.3 Métricas de éxito	7
8. Anexos	7
8.1 Fragmento de host.json	7
8.2 Fragmento de local.settings.json.example (placeholders)	7
8.3 Fragmento de start-local.ps1	8
8.4 Ejemplo de function.json y __init__.py stub	8
8.5 Referencias internas	8
9. Información faltante para completar	9

1. Objetivos y público

1.1 Objetivos del SaaS

- Detectar y priorizar riesgos en flujos de telemetría de seguridad.
- Automatizar respuesta operativa con eventos y playbooks.
- Reducir tiempo de diagnóstico mediante explicaciones asistidas.
- Habilitar despliegue incremental con feature flags por módulo.

1.2 Público objetivo

- SOC: analistas de alertas y correlación de incidentes.
- NOC: operación y monitoreo de plataforma.
- DevOps/SRE: despliegue, CI/CD y observabilidad.
- Integradores: conexión con SIEM, data lake y colas/eventos.
- CISO/arquitectura: gobierno, riesgo y cumplimiento.

1.3 Casos de uso prioritarios

- Validación local de endpoints de auditoría y publicación con stubs.
- Integración de telemetría enriquecida para scoring de riesgo.
- Automatización de notificaciones internas y consumidores de eventos.
- Activación progresiva de módulos de IA y deception con controles.

2. Funcionalidades y utilidades

2.1 Catálogo funcional

Detección predictiva

- Qué hace: estima riesgo sobre telemetría histórica y reciente.
- Cómo se usa: ingesta periódica + reglas/umbral por severidad.
- Endpoints relevantes: /api/extract_telemetry, /api/run_inference.
- Input/Output: JSON de eventos -> JSON de riesgo y clasificación.
- Métricas clave: tasa de detección, precisión, falsos positivos.

Análisis de logs en tiempo real

- Qué hace: procesa señales de actividad para detección temprana.
- Cómo se usa: flujo continuo hacia canal de análisis.
- Endpoints: /api/extract_telemetry.
- Input/Output: payload observacional -> resumen estructurado.
- Métricas: latencia de análisis, throughput, pérdida de eventos.

Alertas internas

- Qué hace: emite eventos y señales operativas para respuesta.
- Cómo se usa: publicación a bus/evento para consumidores.
- Endpoints: /api/publish_event.
- Input/Output: evento normalizado -> confirmación de publicación.
- Métricas: tiempo de entrega, ratio de reintentos, error rate.

Deception / honeypots (apagado por defecto)

- Qué hace: captura tácticas de intrusión en entornos aislados.
- Cómo se usa: activar con ENABLE_DECEPTION=true solo tras revisión legal.
- Endpoints: <PLACEHOLDER_DECEPTION_ENDPOINTS>.
- Input/Output: tráfico señuelo -> indicadores de compromiso.
- Métricas: tasa de interacción maliciosa, IOC útiles, ruido.

Zero-trust dinámico

- Qué hace: endurece acceso por contexto de riesgo y verificación continua.
- Cómo se usa: políticas por identidad/servicio y evaluación continua.
- Endpoints: copilot_adapter y endpoints de control (placeholder).
- Input/Output: atributos de contexto -> decisión de acceso.
- Métricas: bloqueos efectivos, fricción operativa, cobertura de políticas.

Monitorización de accesos

- Qué hace: traza y audita acceso a funciones/componentes críticos.
- Cómo se usa: logs estructurados + correlación por identidad.
- Endpoints: todos los HTTP triggers stubbed.
- Input/Output: metadatos de request -> eventos de auditoría.
- Métricas: cobertura de auditoría, integridad de trazas.

Telemetría enriquecida

- Qué hace: agrega contexto para inferencia y priorización.
- Cómo se usa: pipeline de enriquecimiento antes de inferencia.
- Endpoints: /api/extract_telemetry, /api/run_inference.
- Input/Output: evento base -> evento enriquecido.
- Métricas: calidad de campos, completitud, valor predictivo.

Copilot Runtime

- Qué hace: entrega explicaciones y contexto operacional para equipos.
- Cómo se usa: adapter de consulta con guardrails y políticas.
- Endpoints: /api/copilot_adapter.
- Input/Output: prompt controlado -> respuesta explicativa.
- Métricas: tiempo de respuesta, utilidad percibida, tasa de rechazo.

APIs de IA nativas

- Qué hace: integra modelos cloud/local según políticas y coste.
- Cómo se usa: feature flag ENABLE_LLM y control de cuota.
- Endpoints: /api/run_inference, /api/copilot_adapter.
- Input/Output: contexto + prompt -> inferencia estructurada.
- Métricas: coste por día, latencia, exactitud esperada.

Integración con Azure OpenAI y modelos locales

- Qué hace: permite alternar backend de inferencia.
- Cómo se usa: secretos en Key Vault y fallback local.
- Endpoints: /api/run_inference (placeholder de runtime real).
- Input/Output: solicitud inferencia -> resultado con metadatos.
- Métricas: disponibilidad, coste unitario, calidad de salida.

Sandbox seguro para apps

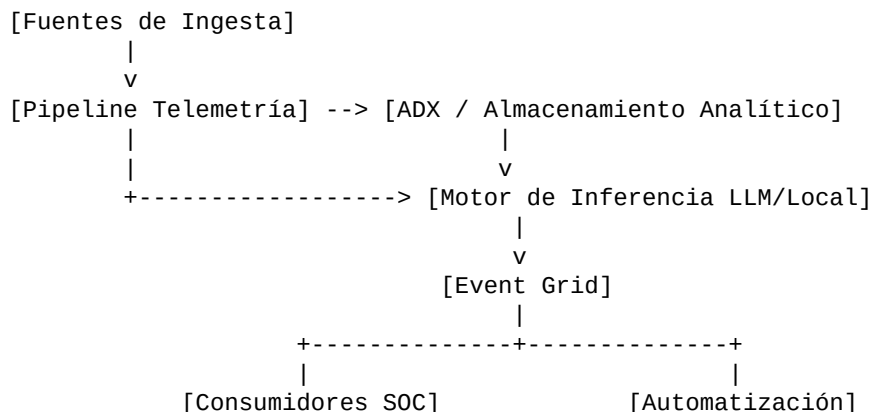
- Qué hace: aísla pruebas y validaciones sin impacto productivo.
- Cómo se usa: staging con mocks y egress controlado.
- Endpoints: todos en modo stub/local.
- Métricas: incidentes de escape, cobertura de casos.

Orquestación y automatización

- Qué hace: coordina reglas y acciones operativas multi-módulo.
- Cómo se usa: playbooks disparados por eventos.
- Endpoints: /api/publish_event.
- Métricas: MTTR, tiempo de cierre, automatizaciones exitosas.

3. Arquitectura y tecnología

3.1 Diagrama textual



3.2 Stack tecnológico

- Azure Functions (Python) para endpoints y ejecución serverless.
- Azure Data Explorer (ADX) para analítica/consulta de telemetría.
- Event Grid para propagación de eventos.
- Azure OpenAI (opcional) y modelos locales para inferencia.
- Bicep IaC para infraestructura (ruta infra/parameters).
- Azurite (dev) para emulación de storage.
- Python con entorno .venv.
- Seguridad de ciclo: Semgrep, pip-audit, OWASP ZAP (pendiente evidencia local).
- Generación de PDF local: reportlab.

3.3 Integraciones y adaptadores

- adx_client: <PLACEHOLDER_IMPLEMENTACION>
- eventgrid_client: <PLACEHOLDER_IMPLEMENTACION>
- openai_client: <PLACEHOLDER_IMPLEMENTACION>
- copilot_adapter: endpoint stub disponible en /api/copilot_adapter.

4. Seguridad, pruebas y evidencia

4.1 Evidencia SAST/SCA/DAST disponible

- Pipeline CI configurado en .github/workflows/ci-security.yml.
- Script de bloqueo CVSS>=9.0 en .github/workflows/scripts/check_pip_audit_critical.py.

Evidencia faltante en release/security-reports (no encontrada localmente):

- release/security-reports/semgrep.json
- release/security-reports/zap_report.json
- release/security-reports/pip-audit.json (consolidado de ejecución)

4.2 Smoke tests y E2E (estado local)

Pasos reproducibles:

- Activar .venv.
- Arrancar host local con func start --no-bundles.
- Probar POST /api/extract_telemetry y POST /api/publish_event.

Resultado observado con stubs:

- HTTP 200 OK.
- Respuesta JSON tipo: {"stubbed": true, "function": "extract_telemetry|publish_event", "received": {...}}.

4.3 Riesgos críticos y mitigación priorizada

- Secretos en configuración no controlada.
- Mitigación: Key Vault + Managed Identity, prohibir secretos en repo.
- Dependencias vulnerables.
- Mitigación: pip-audit obligatorio, bloqueo CVSS>=9.0.
- Endpoints públicos sin endurecimiento.
- Mitigación: auth y validación de payload por entorno.
- Riesgo legal/operativo de deception.
- Mitigación: revisión legal, aislamiento de red, logging segregado.

4.4 Runbook de secretos y checklist hardening

Referencia principal: docs/security-runbook.md.

Checklist:

- [] local.settings.json fuera de Git.
- [] Secretos solo en Key Vault.
- [] Managed Identity habilitada.
- [] Rotación periódica de secretos.
- [] Alertas y auditoría de acceso a secretos.
- [] ENABLE_LLM=false y ENABLE_DECEPTION=false en pruebas iniciales.

5. Cómo aplicar en una empresa

5.1 Guía de integración por etapas

- Staging con mocks y feature flags conservadoras.
- Canary con tráfico controlado y límites de coste.
- Producción gradual con observabilidad y rollback definidos.

5.2 Requisitos operativos y roles

- CISO: aprobación de riesgo y cumplimiento.
- SRE/DevOps: CI/CD, disponibilidad, rollback.
- SOC: reglas de priorización y respuesta.
- Legal/Compliance: políticas de deception y privacidad.

5.3 Políticas de deception

- Activar solo tras aprobación legal documentada.
- Aislar honeypots en subredes dedicadas.
- Registrar y retener evidencia en almacenamiento segregado.

6. Diferenciadores e innovación

- IA nativa + zero-trust dinámico para priorización adaptativa.
- Copilot Runtime para explicación operativa contextual.
- Arquitectura modular con feature flags por dominio.
- Deception integrada (apagada por defecto) con enfoque controlado.
- Telemetría enriquecida orientada a predicción y automatización.

Beneficios medibles esperados:

- Reducción de MTTR.
- Menor fatiga de alertas por priorización automática.
- Ahorro de coste al desacoplar mocks/LLM real por etapa.

7. Plan de despliegue y roadmap corto plazo

7.1 Checklist staging/producción

- [] Validación CI en verde.
- [] Reportes de seguridad adjuntos en release/security-reports.
- [] Secretos en Key Vault y MI funcional.
- [] Smoke tests E2E aprobados.
- [] Plan de rollback documentado.

7.2 Roadmap 2 semanas

Referencia: docs/plan-modular-2-semanas.md.

- Modularización services/security, services/automation, services/deception.
- Cobertura de tests unitarios por módulo.
- Frontend mínimo de operación/observabilidad (placeholder).
- Integración completa de secretos y gating de despliegue.

7.3 Métricas de éxito

- Disponibilidad de endpoints críticos.
- Cobertura de pruebas y tasa de fallos.
- Tiempo de respuesta a incidentes (MTTR).
- Costo diario de inferencia y cumplimiento de cuota.

8. Anexos

8.1 Fragmento de host.json

```
{
  "version": "2.0",
  "logging": {
    "applicationInsights": {
      "samplingSettings": {
        "isEnabled": false
      }
    }
  }
}
```

8.2 Fragmento de local.settings.json.example (placeholders)

```
{
  "IsEncrypted": false,
  "Values": {
    "FUNCTIONS_WORKER_RUNTIME": "python",
    "ENABLE_SECURITY": "true",
    "ENABLE_AUTOMATION": "true",
    "ENABLE_DECEPTION": "false",
    "ENABLE_LLM": "false",
    "AZURE_OPENAI_ENDPOINT": "<AZURE_OPENAI_ENDPOINT_PLACEHOLDER>",
    "AZURE_OPENAI_KEY": "<KEY_IN_KEYVAULT_PLACEHOLDER>",
    "ADX_CLUSTER": "<ADX_CLUSTER_PLACEHOLDER>",
    "ADX_DATABASE": "<ADX_DATABASE_PLACEHOLDER>"
  }
}
```

8.3 Fragmento de start-local.ps1

```
Set-Location "$PSScriptRoot"
if (Test-Path .\venv\Scripts\Activate.ps1) {
  & .\venv\Scripts\Activate.ps1
} elseif (Test-Path .\venv-1\Scripts\Activate.ps1) {
  & .\venv-1\Scripts\Activate.ps1
}
func start --no-bundles
```

8.4 Ejemplo de function.json y __init__.py stub

```

{
  "bindings": [
    {
      "authLevel": "anonymous",
      "type": "httpTrigger",
      "direction": "in",
      "name": "req",
      "methods": ["post"]
    },
    {
      "type": "http",
      "direction": "out",
      "name": "$return"
    }
  ],
  "scriptFile": "__init__.py"
}

import json
import azure.functions as func

def main(req: func.HttpRequest) -> func.HttpResponse:
    try:
        body = req.get_json()
    except Exception:
        body = {}
    payload = {"stubbed": True, "function": "extract_telemetry", "received": body}
    return func.HttpResponse(json.dumps(payload), status_code=200, mimetype="application/json")

```

8.5 Referencias internas

- README.md
- local.settings.json.example
- start-local.ps1
- host.json
- .github/workflows/ci-security.yml
- .github/workflows/scripts/check_pip_audit_critical.py
- docs/security-runbook.md
- docs/plan-modular-2-semanas.md

9. Información faltante para completar

Archivos o datos no encontrados localmente:

- release/security-reports/semgrep.json
- release/security-reports/zap_report.json
- release/security-reports/pip-audit.json (ejecución exportada)
- migration_report_azure.json
- Payloads reales anonimizados para validación E2E avanzada

Notas de cumplimiento:

- Este dossier no incluye secretos reales.
- Todos los valores sensibles se representaron con placeholders y referencias a Key Vault.